The MABEL 96xx Data Plane: Dedicated-Port Streaming of Pointclouds, SLAM, Proprioception, and Teleoperation Telemetry for Sim and Real

MABEL project — `server/`

November 4, 2026

Abstract

The MABEL teleoperation server historically exposed one control WebSocket (`:9090`), one multiplexed MJPEG camera server (`:8080`), and two UDP video paths — sufficient for driving the robot, but under-reporting what the system measures: `robot_state` carried joint *positions* only, the battery block was a hard-coded constant, e-stop and control-mode state were never echoed, and the pointclouds, nvblox mesh, and cuVSLAM pose that already exist on the real robot's ROS graph never reached a client. This document describes the *96xx data plane*: ten additional read-only broadcast listeners — typed-JSON telemetry (`:9600`), four per-camera pointcloud ports (`:9601`–`:9604`), a SLAM mesh+pose+map port (`:9605`), and four dedicated video ports (`:9620`–`:9623`, of which only the side-by-side stereo head stream is high-resolution). The plane is strictly additive (commands stay on `:9090`; every legacy byte is unchanged), holds identical port numbers in simulation and on the physical robot, reuses the existing binary map-stream record format so client decoders need only a port change, and degrades to explicit machine-readable *notices* when a source is absent in the current world. Measured on the Mac simulation stack: telemetry sustains ∼46 Hz proprioception under all load combinations, each cloud port delivers 12–18k points per frame at the 2 Hz design rate, and the hi-res stereo stream sustains 12–17.8 fps at 1920 × 720; the only contended resource is the serialized macOS GL render slot, quantified below.

Keywords: teleoperation, telemetry, pointcloud streaming, nvblox, cuVSLAM, WebSocket, MJPEG, sim-to-real parity, demand-driven rendering.

1 Overview and design constraints

Four constraints shaped the design, in priority order:

1. **Additive only.** The existing architecture works and is depended on by three client apps; nothing about `:9090`/`:8080`/`:9091`/`:9092` may change shape. All new data rides new ports; the only `:9090` changes are *additive* JSON keys old clients ignore.
2. **Minimal client churn.** New planes reuse formats clients already decode: the binary records of `MAP_STREAM_PROTOCOL.md` (CLOUD 0x05, MESH 0x01, ODOM 0x02, GRID 0x04) and the `/camera/<name>/stream.mjpg` MJPEG path shape. Consuming any new stream is a port change.
3. **Sim–real parity.** Port numbers are identical in both worlds; only the *source* behind a port differs (MuJoCo depth render vs. RealSense `PointCloud2`; `qfrc_actuator` vs. motor-driver effort). A source absent in the current world binds its port anyway and answers with an explicit notice — an explanation, never a refused connection.

4. **The control loop is sacred.** No new work may slow the real-time physics/control thread. Every producer is demand-gated (an unwatched robot pays nothing), state hand-off is by reference-swapped dicts, all queues are newest-wins, and heavy encoding happens on the network thread or in the separate camera process.

2 The port contract

Table 1 is the master contract (mirrored in `DATA_STREAMS.md`). The master gate `MABEL_DATA_STREAMS=0` removes every 96xx listener; verification confirms zero behavioral delta in that mode.

Port	Plane	Content / rate	SIM source	REAL source
9600	telemetry WS	proprio+control 50 Hz; status 5, lidar 5, misc 1 Hz	`qpos/qvel/qfrc`; `mj_multiRay` lidar	`joint_states` vel/eff; hand temps; `/scan`
9601	cloud WS	head, CLOUD 0x05, 2 Hz	depth render (zed)	ZED-M cloud
9602	cloud WS	wrist_left	depth render	D405 cloud
9603	cloud WS	wrist_right	depth render	D405 cloud
9604	cloud WS	front/base	depth render	D435i cloud
9605	SLAM WS	MESH+ODOM+GRID+status	*notice* (no SLAM)	nvblox+cuVSLAM+`/map`
9620	video	head SBS, **hi-res**	1920×720 composite	native SBS frame
9621–23	video	wrists + front, std res	tee of :8080 JPEGs	HAL tee

Table 1: The 96xx plane. Binder processes differ by world (sim: camera server owns 9601–9604/9620–9623, bridge owns 9600/9605; real: bridge owns 9600–9605 because `rclpy` lives there, the camera HAL owns 9620–9623) — invisible to clients. All WS ports enforce the same `?key=` token as `/teleop`. Every plane is LAN/Tailscale-direct.

3 Method

3.1 Telemetry plane (`:9600`)

The hub (`telemetry_stream.py`) separates *producers* from *broadcasters*. On the control thread, a producer runs at most every 20 ms *and only while a subscriber exists* (`TELEMETRY.enabled()`, the proven `MONITOR.enabled()` pattern), building plain dicts and reference-swapping them into versioned slots — identical discipline to the existing `_robot_state` hand-off. Because physics substeps keep pace even when the outer control iteration drops to ∼35 Hz under load, the producer is invoked from the *substep* block (the same reasoning that moved the `robot_state` refresh there), with the outer loop as fallback. On the network thread, three broadcaster coroutines (50/5/1 Hz classes) JSON-encode each new snapshot *once* and fan out to per-socket latest-wins queues (`maxsize`=8, drop-oldest); a slow client gets fresher data, never a backlog. On the real robot, proprioception is produced by the existing decoupled 60 Hz feedback thread from `plant.joint_dynamics()` — a new accessor over the hardware `joint_states` velocity/effort arrays and the hands' temperature topics, cached with the same atomic-rebind discipline in `ros_plant.py`.

Missing measurements are `null`, and missing subsystems are {`available:false, reason`} — the battery (no BMS exists in either world), simulated temperatures (MuJoCo has none), and arm temperatures (the DaMiao driver measures them, bytes 6–7 of the CAN feedback, but the ROS node does not yet parse them) are all reported honestly rather than guessed.

3.2 Sim pointclouds: internal depth render + back-projection

The real robot publishes `PointCloud2` directly; the Mac sim has no depth sensor, so `cloud_streams.SimDepthCloudSource` renders depth *internally* (never on the wire) with one dedicated thread owning one GL context (the `sim_sensor_source.py` pattern), round-robining only the subscribed cameras at 2 Hz each, 320×240. A MuJoCo camera is a pinhole with vertical field-of-view ϕ; at render height h the focal length is

$$f \;=\; \frac{h}{2\,\tan(\phi/2)}, \qquad \mathbf{p}_{\text{cam}} = \begin{bmatrix} (u - c_x)\, z/f \\ -(v - c_y)\, z/f \\ -z \end{bmatrix}, \qquad \mathbf{p}_{\text{world}} = R_{\text{cam}}\, \mathbf{p}_{\text{cam}} + \mathbf{t}_{\text{cam}},$$

where z is the planar depth returned by the renderer, (u, v) the pixel with v growing downward (hence the sign flip: the MuJoCo/OpenGL camera has $+y$ up and looks along $-z$), and $(R_{\text{cam}}, \mathbf{t}_{\text{cam}})$ come from `data.cam_xmat`/`cam_xpos` after `mj_kinematics`. Validity of this mapping was unit-tested against a known scene: a planar wall at 2.0 m (front face 1.95 m) reconstructs at $z = -1.9500 \pm 0.0000$ m with the lateral extent matching the 60° fovy prediction to < 5 cm. The pixel grid is strided so a full frame lands under the 20k-point cap; rangefinder visualization geoms are disabled in the render options (they poison depth). RGB is rendered at the same pose for per-point color. On the real robot, `RosCloudSource` gives each camera topic its *own* DDS participant (one UDP socket per firehose — the map-stream lesson: a 30 Hz ∼0.5 MB/frame depth stream sharing a socket with RELIABLE mesh samples livelocked retransmissions), subscribes only while its port has a client, and transforms clouds into the `odom` frame via TF when available.

3.3 SLAM plane (`:9605`) and dedicated video (`:962x`)

The SLAM plane is a thin wrapper (`slam_stream.py`) around a second `MapRosSource` instance — reusing its QoS matching, TF handling, nvblox block accumulation, and rate caps — pointed at the cuVSLAM *tracking* odometry (`/visual_slam/tracking/odometry`) rather than the fused wheel `/odom` the `:9092` map plane ships, plus the `/map` occupancy grid, with a 1 Hz JSON status frame so a client can distinguish "no mesh yet" from "SLAM is down." In sim it serves a notice: SLAM runs only on the robot.

Feeding the legacy map plane in sim. The iOS and Vision Pro Maps tabs render the pre-existing `:9092 /map` plane, which on the Mac had *no producer* (its source is a ROS subscriber). A bridge-local task (`_sim_map_feed`) now feeds it in sim — the robot pose (ODOM 0x02 from the base state) plus one live CLOUD 0x05 that merges the front-camera pointcloud (relayed over localhost from the `:9604` plane, demand-gated so the depth renderer idles when no Maps tab is open) with the lidar sweep's world-frame hit points drawn as a red ring — so pointclouds and laser data appear in the apps' existing 3-D map view with *zero* client changes (measured: ODOM + CLOUD at ∼9.1k points/frame). On the real robot the ROS source feeds the same plane and the task is skipped.

Dedicated video gives each camera its own HTTP listener so one stream's burst (or one slow client) never contends with another in the `:8080` mux. Only the head is high-resolution: *one* side-by-side stereo MJPEG — matching the real head camera, which natively emits a hardware-stitched SBS frame — composited in sim from the two eye renders (`np.hstack`, encoded once at quality 88, only while `:9620` has a client). While the SBS stream is watched, both eyes render uncapped at the head cadence; a secondary-fps-capped right eye would give the stereo pair mismatched timestamps. Wrist and front ports tee the already-encoded standard-resolution JPEGs (zero extra GPU or encode). MJPEG was chosen over H.264 for these ports because the Mac's VideoToolbox pipeline

adds ∼400 ms fixed latency (measured previously) while MJPEG drop-to-latest is 3–8 ms; the real robot's NVENC H.264 on `:9092` is unaffected.

4 Experiments

All numbers from `tools/exp_data_streams.py` against the live stack (`./run.sh --no-monitor`, `mabel_home` scene, M-series Mac, 6 s windows; 34/34 automated checks pass). The harness is the reproduction path: `--probe` re-runs every check, `--contention` the load matrix, `--figures` regenerates Figures 1–3.

Plane	Rate	Payload	Bandwidth
telemetry proprio / control	46.3 / 46.2 Hz	54 joints × {q,dq,tau,temp}	∼0.4 MB/s
telemetry status / lidar / network	5.2 / 5.0 / 1.2 Hz	full status; 360-ray sweep	negligible
telemetry ping RTT (loopback)	—	—	0.8 ms
cloud head / front	2.0 fr/s	16.1k / 17.8k pts	0.52 / 0.57 MB/s
cloud wrist L / R	2.0 fr/s	12.0k / 12.0k pts	0.38 / 0.39 MB/s
video head SBS (1920 × 720, q88)	12–17.8 fps	∼120 kB/frame	1.4–2.2 MB/s
video wrists / front (480 × 360)	∼5.9 fps	tee of :8080	0.10–0.14 MB/s

Table 2: Measured steady-state performance per plane (sim). The SBS range reflects ambient GL load: 17.8 fps with only the SBS client attached, 12 fps during the full probe. Cloud points are post range-mask, pre-encode-cap.

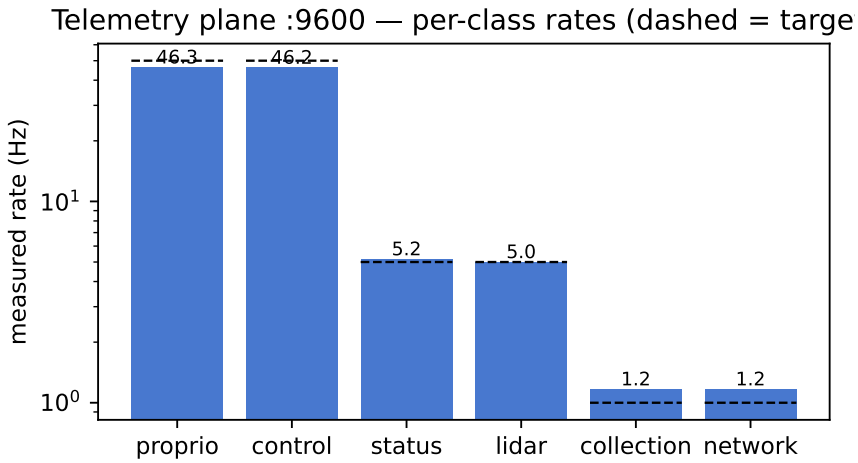


Figure 1: Telemetry per-class rates vs. their targets (log scale). The 50 Hz classes settle at ∼46 Hz: the 20 ms producer gate beats against the 4 ms physics substep cadence, so a tick occasionally lands at 24 ms — a bounded, honest cost we accept rather than moving production off the thread that owns `MjData`.

Three additional verified properties: (i) **kill switch** — `MABEL_DATA_STREAMS=0` leaves zero 96xx listeners and an unchanged legacy plane; (ii) **back-compat** — every legacy `robot_state` key is byte-shape-identical, the additive keys (`estop`, `controlMode`, `torque`, `dataCollection`, `policy`, `animation`, `battery.available`) are present, and `hello` carries the 14-entry port map; (iii) **world-frame sanity** — all four sim clouds reconstruct within the scene bounds ($\max\|p\| \leq 4.4$ m from the base).

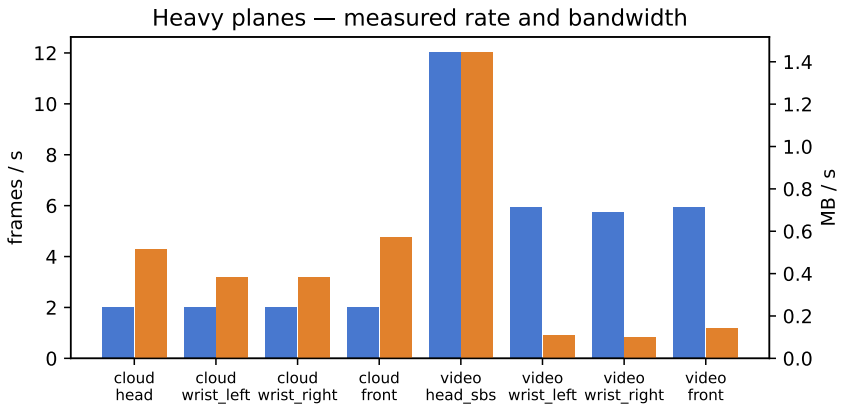


Figure 2: Per-stream frame rate and bandwidth for the heavy planes during the full probe.

5 Implementation and usage

```
cd server && ./run.sh                          # everything, data plane on by default
MABEL_DATA_STREAMS=0 ./run.sh                  # legacy behavior, zero new listeners
python tools/exp_data_streams.py --probe       # 34-check verification vs the live stack
python tools/exp_data_streams.py --figures     # regenerate docs/figures/ds_*.pdf
```

New modules: `data_streams.py` (ports, gating, generic token-gated dedicated-port server, notices), `telemetry_stream.py`, `cloud_streams.py`, `slam_stream.py`, `dedicated_video.py`. Modified: the bridge (payload keys, handlers, producers, serve), `plant.py`/`ros_plant.py` (dynamics accessors), `sim_camera_server.py`/`real_camera_server.py` (plane hosting), `hardware_bridge` (per-port `VideoServer` tee, both-wrists-D405 config with serial claim dedupe), and the launchers. Env knobs: `MABEL_CLOUD_{HZ,RES,MAX_POINTS,MAX_RANGE}`, `MABEL_SBS_QUALITY`, `MABEL_POLICY_DIR`, `MABEL_ANIMATION_DIR`, per-port overrides in `data_streams.py`.

6 Limitations

- **GL budget (sim).** All off-screen renders share one serialized ∼41–54 ms macOS readback slot; with every plane subscribed the legacy head stream drops to 8.6 fps (Fig. 3). Clouds are capped at 2 Hz/camera for this reason; raising `MABEL_CLOUD_HZ` spends head fps.
- **50 Hz classes deliver ∼46 Hz** (gate/substep beat); if exact 50 Hz matters a monotonic-deadline producer would fix it at the cost of occasional double work per outer iteration.
- **Real-robot claims are code-complete but bench-unverified**: the thor path (ROS clouds, SLAM plane, hand temperatures, both-D405 wrists, HAL video tee) compiles and mirrors verified sim seams, but only some sensors are currently connected; the on-robot checklist in `DATA_STREAMS.md` governs sign-off. Unpinned dual-D405 side assignment follows bring-up order — pin `CAM_WRIST_{LEFT,RIGHT}_SERIAL` for determinism.
- **Placeholders by design**: battery (`available:false` until a BMS exists), arm temperatures (ROS driver does not parse DaMiao bytes 6–7 yet), policy/animation (selection echo only; a runner would plug into the `RobotCommand` seam).
- **Direct links only**: remote consumption requires Tailscale.

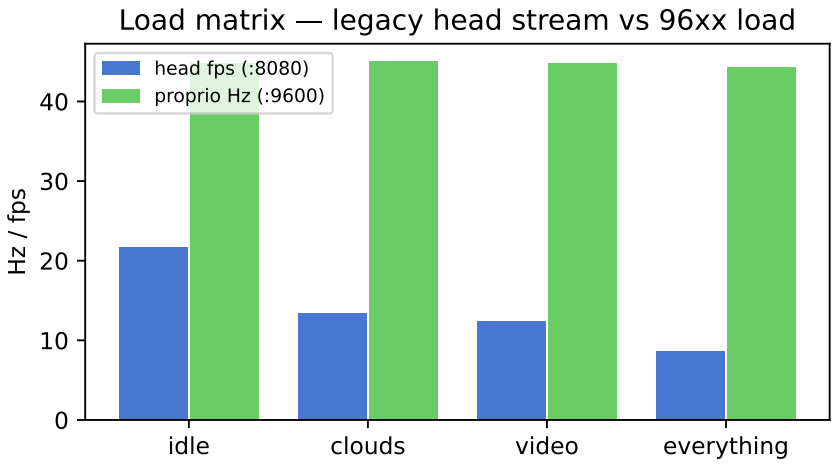


Figure 3: Load matrix: the legacy `:8080` head stream and the `:9600` proprioception rate as 96xx subscriptions accumulate. Head fps degrades 21.6 → 8.6 as clouds + dedicated video join (all renderers share one serialized macOS GL readback slot, ∼24 slots/s) — *but proprioception holds ∼44–45 Hz in every scenario*: the control loop is insulated from the GL budget by design.

References

[1] A. Millane et al., "nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping," *IEEE ICRA*, 2024.

[2] NVIDIA Isaac ROS, "cuVSLAM: CUDA Visual SLAM," https://nvidia-isaac-ros.github.io, 2024.

[3] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE/RSJ IROS*, 2012.

[4] MABEL project, "The MABEL Teleoperation Protocol," `server/docs/teleop_protocol.pdf`, 2026.

[5] MABEL project, "The MABEL Teleop Server," `server/docs/server.pdf`, 2026.